Journey Finder App

(prototype)

Mohmmed Azimur Rahman Chowdhury

JOURNEY FINDER

YOUR JOURNEY IS REWARDED

Contents

- Introduction
- Changes and design
- Concepts & Technologies
- Firebase
- Login Page
- Sign-Up Page
- Forgot Password Page
- Home Page
- Maps Page
- Profile Page
- Community Page
- Challenges
- Future Improvements
- References

Introduction

- The original idea for this app was to create a platform where tourists can easily find nearby locations to visit, be it restaurants, shops, events and landmarks. They'll also be able share their experience through the in-app "social media".
- The idea was to help bring eyes to smaller businesses (shops) and help develop local towns.

- The whole app will be developed using android studio (Java).

Changes and design

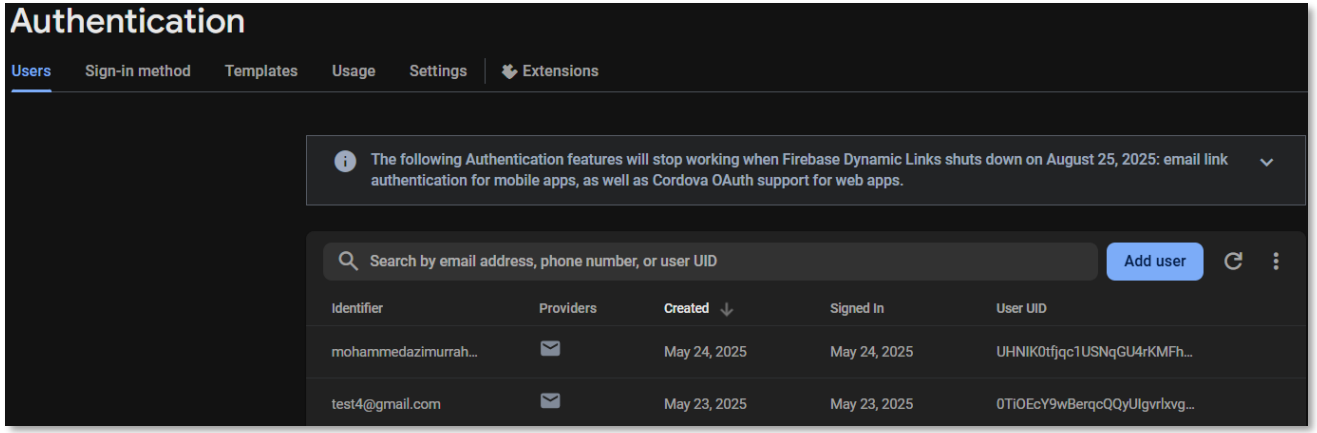
- Before development SQLite was going to used along side Firebase, but due to complications and time constraints, this was scrapped and only Firebase Authentication and Firestore was used, this allowed for better security and real time capabilities for the community page.
- Another change was not implementing a payment method as proposed within the first submission, as well as a reward system. The reason for this is due to being overly confident when coming up with the idea and not having the time to implement them effectively.

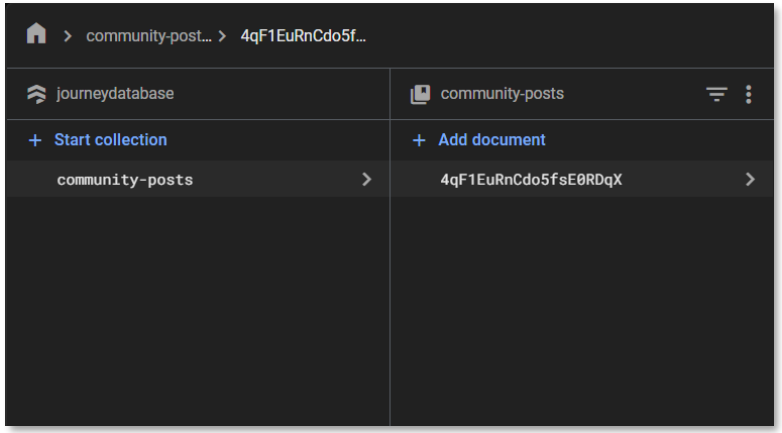
Concepts & Technologies

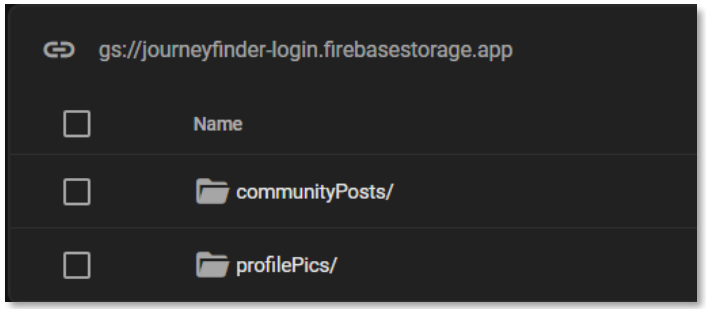
- Android Studio (Java)
- Firebase authentication
- Firebase Firestore (real time database)
- Firebase Storage
- Google maps SDK
- Google Places API
- Session management
- RecyclerView
- Adapters
- XML layouts
- Intents

Firebase

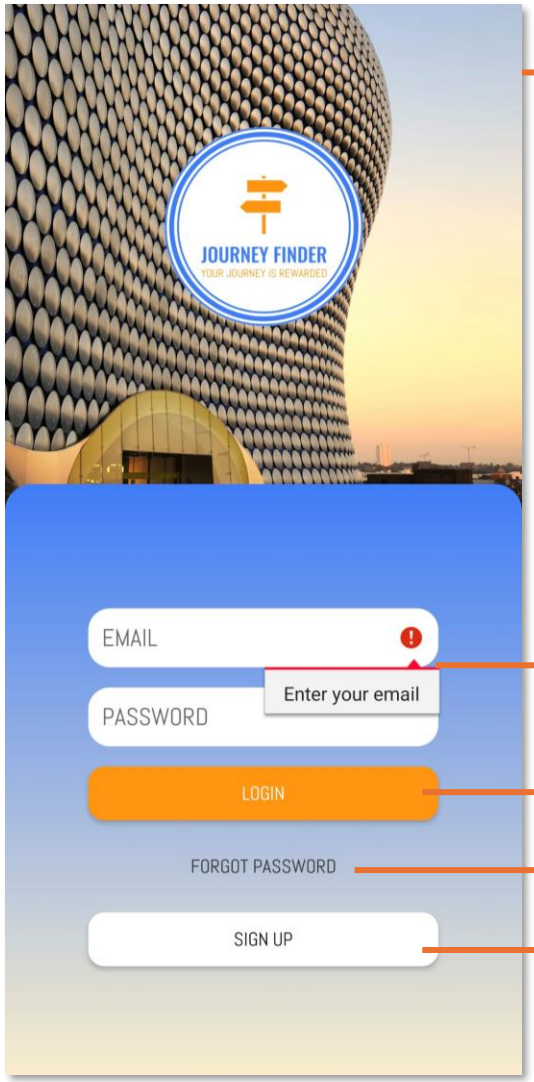
- Firebase was the main data persistence used for this project. It takes advantage of authentication for user login details, database to store real time community posts and storage to store user profile images and post images.







Login Page



As the user boots up the application they will be met by the login page.

To access the apps features the users will have to login using their email and password. This will be handled using Firebase authentication.

The “Login” button will direct the user to the “home” page once the fields have been filled correctly.

If the user has forgotten their password, they can reset it. This will be processed through Firebase password rest.

If the user does not have an account, they can easily create one pressing the “Sign up” button.

Login Code - 1

As to make the application seamless to use, persistent session is implemented. It checks if there is a user logged in, if the “currentuser” is not null, that mean the user can be redirected to the home page upon launching.

“mAuth” initialises the Firebase authentication. Also linking the variables to the XML IDs. This’ll allow the buttons to work, through listener, so when clicked “loginUser” is called.

```java
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    //Checking if the user has been logged in
    FirebaseUser currentUser = FirebaseAuth.getInstance().getCurrentUser();
    if (currentUser != null) {
        startActivity(new Intent( packageContext: LoginActivity.this, HomeActivity.class));
        finish();
    }
    setContentView(R.layout.activity_login);

    //Initialise firebase
    mAuth = FirebaseAuth.getInstance();
    //variables linked to XML IDs
    login_email = findViewById(R.id.login_email);
    login_pass = findViewById(R.id.login_pass);
    loginButton = findViewById(R.id.loginButton);
    signupButton = findViewById(R.id.signupButton);
    forgotPass = findViewById(R.id.forgotPass);
    //Login
    loginButton.setOnClickListener( View v -> loginUser());
```

```java
private void loginUser() {
    String email = login_email.getText().toString();
    String password = login_pass.getText().toString();

    //Input validation
    if (TextUtils.isEmpty(email)) {
        login_email.setError("Enter your email");
        login_email.requestFocus();
        return;
    }
    if (!Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
        login_email.setError("Enter a valid email");
        login_email.requestFocus();
        return;
    }
    if (TextUtils.isEmpty(password)) {
        login_pass.setError("Enter your password");
        login_pass.requestFocus();
        return;
    }
```

Using “TextUtils” from Android will check if the email and password fields are filled, if not a prompt will appear asking the user to fill it.

To check if the user has inputted a valid format for an email, androids pattern checker is used to see if it’s a valid email.

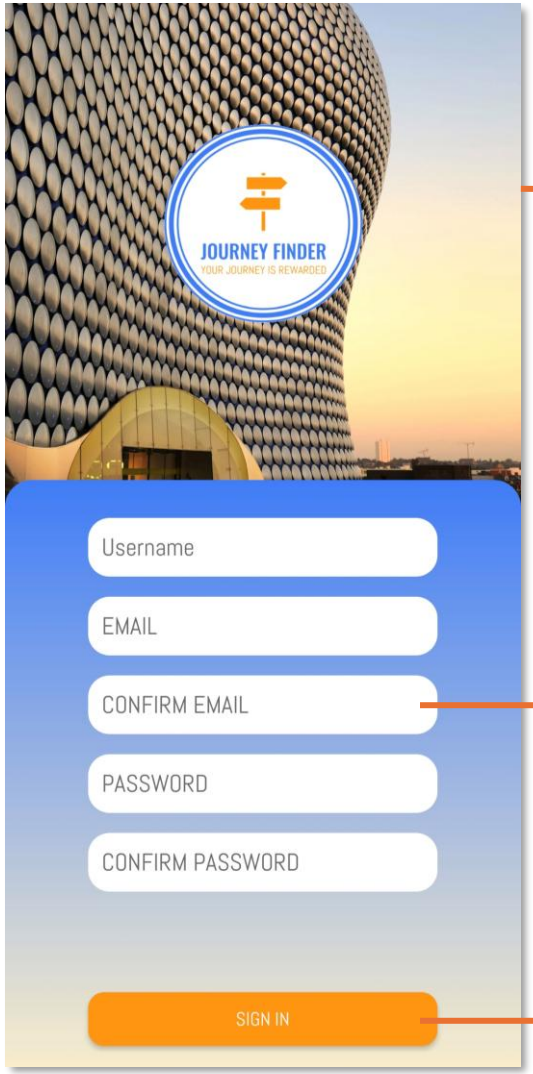
Login Code - 2

The FirebaseAuth is called again to check the user login details and tries to log them in. Using the listener, it checks if the login was a success or not, using “AuthResult”.
If it’s a success the user is logged in and directed to the “Home” page.
If not, using “Toast” the user is notified the login failed.
This was all sourced through (Firebase, n.d.) documentation.

```
//From Firebase login Documentation
mAuth.signInWithEmailAndPassword(email, password)
        .addOnCompleteListener( activity: this, new OnCompleteListener<AuthResult>() {
            @Override
            public void onComplete(@NonNull Task<AuthResult> task) {
                if (task.isSuccessful()) {
                    Log.d( tag: "LOGIN",  msg: "signInWithEmail:success");
                    FirebaseUser user = mAuth.getCurrentUser();
                    Intent intent = new Intent( packageContext: LoginActivity.this, HomeActivity.class);
                    startActivity(intent);
                    finish();

                } else {
                    Log.w( tag: "LOGIN",  msg: "signInWithEmail:failure", task.getException());
                    Toast.makeText( context: LoginActivity.this,  text: "Authentication failed.", Toast.LENGTH_SHORT).show();
                }
            }
        });
```

Sign-Up Page



Very similar design, to keep it consistent and follow UX laws. All learnt from previous module. Upon Pressing the “Sign Up” button within the login page, the user will be taken to this page. Where it prompts the user to fill in the designated fields.

These will have to be filled to create an account, and the confirm boxes are used for validation that the user has inputted what they wanted.

The “Sign In” button, will then redirect the user to the home page.

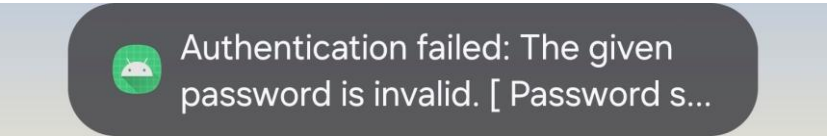
Sign-Up Code - 1

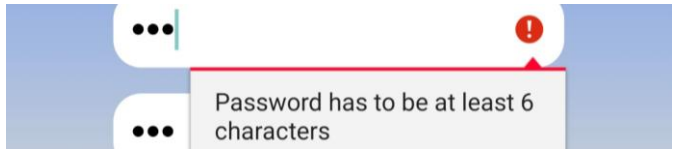
```java
private void registerUser() {
    String name = sign_username.getText().toString();
    String email = sign_email.getText().toString();
    String confirmEmail = confirm_email.getText().toString();
    String password = sign_pass.getText().toString();
    String confirmPassword = confirm_pass.getText().toString();
    //setting username
    if (TextUtils.isEmpty(name)) {
        sign_username.setError("Enter your username");
        sign_username.requestFocus();
        return;
    }
    //checking valid email
    else if (!Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
        sign_email.setError("Enter a valid email");
        return;
    }
    //checks email confirmation
    else if (!email.equals(confirmEmail)) {
        confirm_email.setError("Emails do not match");
        return;
    }
    //sets password minimum length
    if (TextUtils.isEmpty(password)) {
        sign_pass.setError("Enter your password");
        sign_pass.requestFocus();
        return;
    }
    else if (password.length() < 6) {
        sign_pass.setError("Password has to be at least 6 characters");
        return;
    }
    //checks passwords confirmation
    else if (!password.equals(confirmPassword)) {
        confirm_pass.setError("Passwords do not match");
```

Pressing the “Sign up” button will run “registerUser” function. This gets the fields and just like the login code, “TextUtils” was used to check if the fields are being filled. If not “setError” will prompt a message to the user to fill in the empty box or if they do not match.

“Username” will be used to display the users name within the profile page.

Firebase already has a minimum password length, but the error message can be confusing for users so, setting a password length to “< 6” will allow to display an error message for the actual problem.





Sign-Up Code - 2

```java
//Firebase sign in and update user profile
mAuth.createUserWithEmailAndPassword(email, password)
        .addOnCompleteListener( activity: this, new OnCompleteListener<AuthResult>() {
            @Override
            public void onComplete(@NonNull Task<AuthResult> task) {
                if (task.isSuccessful()) {
                    Log.d(TAG,  msg: "createUserWithEmail:success");
                    FirebaseUser user = mAuth.getCurrentUser();

                    //Sets display name
                    if (user != null) {
                        UserProfileChangeRequest profileUpdates = new UserProfileChangeRequest.Builder()
                                .setDisplayName(name)
                                .build();

                        user.updateProfile(profileUpdates)
                                .addOnCompleteListener( Task<Void> profileTask -> {
                                    if (profileTask.isSuccessful()) {
                                        Log.d(TAG,  msg: "User profile updated.");
                                    }
                                });
                    }
                    //takes to the home page
                    startActivity(new Intent( packageContext: SignUpActivity.this, HomeActivity.class));
                    finish();
                } else {
                    Log.w(TAG,  msg: "createUserWithEmail:failure", task.getException());
                    Toast.makeText( context: SignUpActivity.this,  text: "Authentication failed: " + task.getException().getMessage(),
                            Toast.LENGTH_LONG).show();
                }
            }
        });
```

This was done following Googles Firebase documentation. “createUserWithEmailAndPassword” takes the parameters so it can update the Firebase authentication, using “mAuth” (Firebase, 2025).

Since the user had to make a username here, this will have to update the user profile using the “updateProfile”. This was done following Firebase docs again (Firebase, 2025).

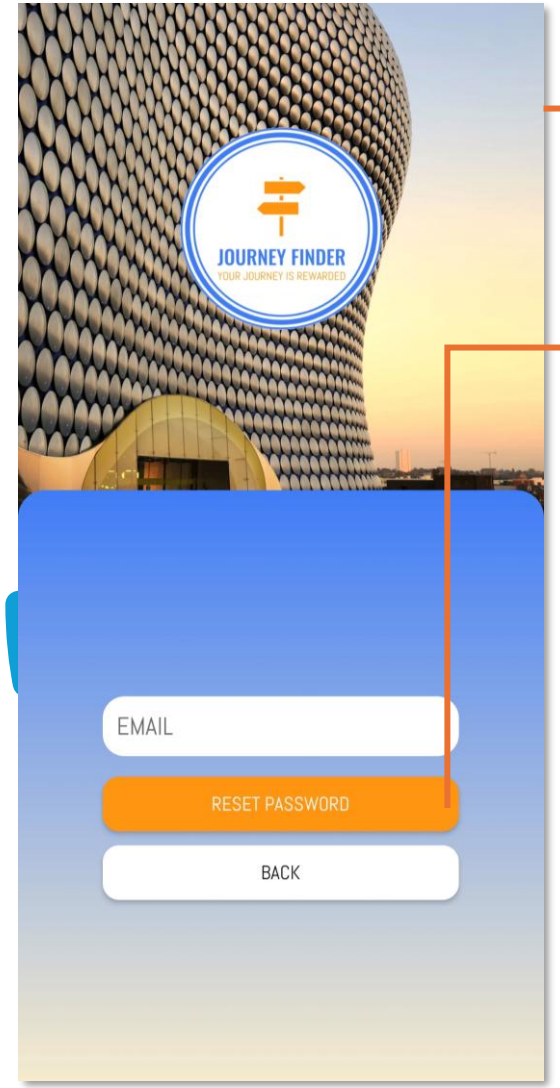
This intent will take the user directly to the home page.

Search by email address, phone number, or user UID | Add user

Identifier	Providers	Created	Signed In	User UID
mohammedazimurrah...		May 24, 2025	May 24, 2025	UHNIK0tfjqc1USNqGU4rKMFh...
test4@gmail.com		May 23, 2025	May 23, 2025	0TiOEcY9wBerqcQQyUIgvrlxvg...

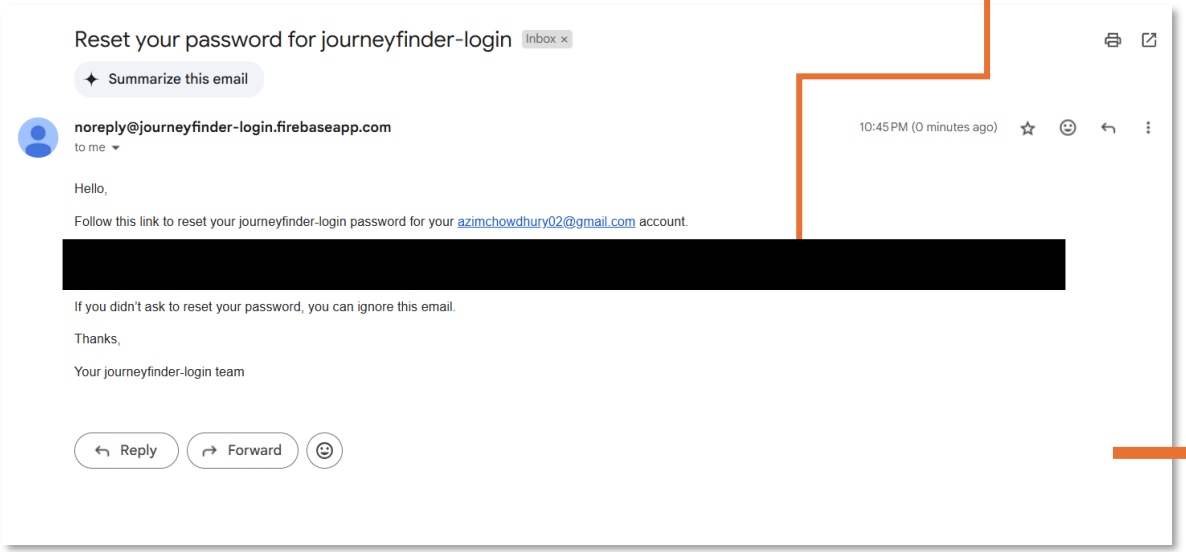
When the user is registered the details will be automatically logged with the authentication database.

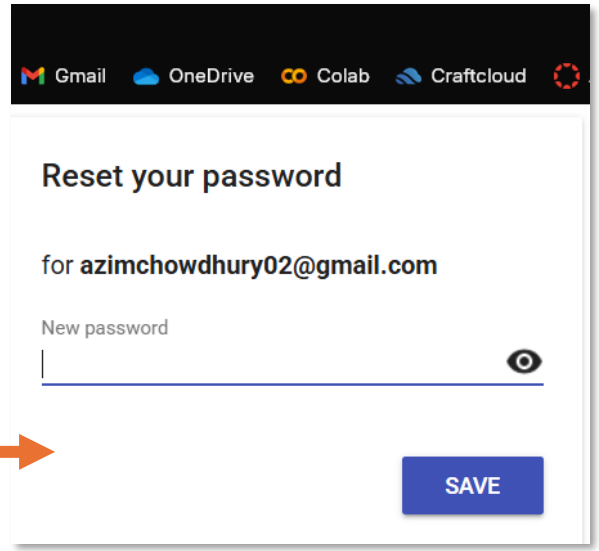
Forgot Password Page



This page is accessed through pressing the “Forgot Password” button, in case of user forgetting their password.

The user will be prompted to enter their email for said account. Upon pressing the “Reset” button the user should receive an email, with a link to reset their password





Forgot Password Code - 1

```java
//Reset password
resetButton.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        strEmail = resetEmailInput.getText().toString();
        if (!TextUtils.isEmpty(strEmail)) {
            resetPassword();
        } else {
            resetEmailInput.setError("Please enter your email");
        }
    }
});
//Back button
backButton.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        onBackPressed();
    }
});
```

"onClick" makes sure that the field is correctly filled, meaning if the user has inputted their email.
If it has been completed the button will call the "resetPassword" method. if not, an error message will be displayed.

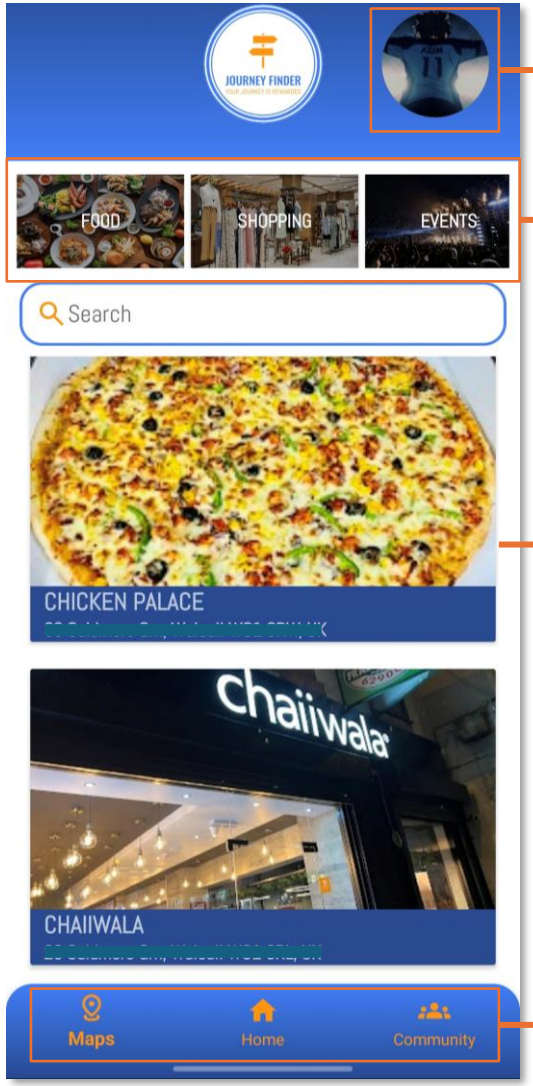
The back button, will take the user pack to the previous page, which will be the login page.

Forgot Password Code - 2

This was also done following (Firebase, 2025) documentation, for “password reset email”. Once the user click the “Rest” button, this method will be called. “sendPasswordRestEmail” is used from “mAuth” to send the reset link to the inputted user email.
Once this is competed the user will be notified with a “Toast” message and then taken back to the login page. Firebase Authentication, will do the rest as shown in Slide.

```
//Firebase reset passwords
1 usage
private void resetPassword() {
    mAuth.sendPasswordResetEmail(strEmail)
            .addOnCompleteListener(new OnCompleteListener<Void>() {
                @Override
                public void onComplete(@NonNull Task<Void> task) {
                    if (task.isSuccessful()) {
                        Toast.makeText( context: ForgotPassActivity.this,
                                text: "Reset link sent to your email", Toast.LENGTH_SHORT).show();
                        startActivity(new Intent( packageContext: ForgotPassActivity.this, LoginActivity.class));
                        finish();
                    }
                }
            });
```

Home Page



The home page will be the first page, for users with accounts, due to persistent sessions.
They will be able to access their **profile page**, other tabs for designated filters (**HorizontalScrollVeiw**), be able to see nearby areas (**RecyclerView**) and the **bottom navigation** for the other pages.

Home Code - 1

```java
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_home);

    //Places API
    if (!Places.isInitialized()) {
        Places.initialize(getApplicationContext(), apiKey: [redacted]);
    }
    placesClient = Places.createClient( context: this);

    //Permission
    if (ContextCompat.checkSelfPermission( context: this, Manifest.permission.ACCESS_FINE_LOCATION)
            != PackageManager.PERMISSION_GRANTED) {
        ActivityCompat.requestPermissions( activity: this,
                new String[]{Manifest.permission.ACCESS_FINE_LOCATION}, requestCode: 2002
        );
    }
```

```java
FusedLocationProviderClient locationClient = LocationServices.getFusedLocationProviderClient( activity: this)
locationClient.getLastLocation().addOnSuccessListener( Location location -> {
    if (location != null) {
        double lat = location.getLatitude();
        double lng = location.getLongitude();
        loadNearbyPlaces(lat, lng);
    }
});
```

Due to the contents in the home page, Google Places API was used to get the places and images that google already have. This will be initialised.

“.PlacesClient” will be used to get the location and images.

Due to needing access to the user's location, permission was needed. The home page shows the closest places to the user, through RecyclerVeiw, so “FINE_LOCATION”, was needed to get most accurate area. And “FusedLocation” is used to get the last known location, then calls “loadNearbyPlaces” for areas.

Home Code - 2

```
// Profile icon
profileIcon = findViewById(R.id.profileIcon);
profileIcon.setOnClickListener( View v -> {
    Intent intent = new Intent( packageContext: HomeActivity.this, ProfileActivity.class);
    startActivity(intent);
});

ImageView profileIcon = findViewById(R.id.profileIcon);
FirebaseUser user = FirebaseAuth.getInstance().getCurrentUser();
if (user != null && user.getPhotoUrl() != null) {
    Glide.with( activity: this) RequestManager
            .load(user.getPhotoUrl()) RequestBuilder<Drawable>
            .placeholder(R.drawable.defaultpfp)
            .circleCrop()
            .into(profileIcon);
}
```

Using “OnClickListener” the user will be able to click and be directed to the profile page, after pressing the profile icon.

Using the FirebaseAuth, this will check if a user is logged in, if there is “!= null” means there is a user, and if they have set a profile image this will be displayed, through using glide, which sets the images from the firebase storage, sets “defautpfp” image, crops it so it fits the in the “ImageVeiw” of “profileIcon”.

Home Code - 3

```java
//Tabs
tabFood = findViewById(R.id.tabFood);
tabShopping = findViewById(R.id.tabShopping);
tabEvents = findViewById(R.id.tabEvents);
tabLandmarks = findViewById(R.id.tabLandmarks);

tabFood.setOnClickListener( View v -> tab( newTab: "restaurant|cafe|bakery|food"));
tabShopping.setOnClickListener( View v -> tab( newTab: "clothing_store|supermarket|furniture_store|" +
        "convenience_store|department_store|electronics_store"));
tabEvents.setOnClickListener( View v -> tab( newTab: "stadium|place_of_worship|hindu_temple|church|" +
        "mosque|zoo|amusement_park|aquarium|night_club"));
tabLandmarks.setOnClickListener( View v -> tab( newTab: "tourist_attraction|landmark|natural_feature|" +
        "art_gallery|park"));

//Bottom nav
bottomNav = findViewById(R.id.bottomNav);
bottomNav.setOnItemSelectedListener( MenuItem item -> {
    int id = item.getItemId();
    if (id == R.id.home_nav) {
        return true;
    } else if (id == R.id.map_nav) {
        startActivity(new Intent( packageContext: HomeActivity.this, MapActivity.class));
        return true;
    } else if (id == R.id.comm_nav) {
        startActivity(new Intent( packageContext: HomeActivity.this, CommActivity.class));
        return true;
    }
    return false;
});
```

Each tab will be assigned a filtered list, when a tab is clicked, it will run the “tab” method, where it uses intents to take the user to the designated page. This page will then show all the relevant places matching the filter set.
Filters were gathered from places types (Google for Developers, 2025).

The bottom navigation is done using intents, each icon is set to the designated page. This will be done for the “Profile”, “Tabs” and “Community” pages

Home Code - 4

```java
//looks for nearby places information
1 usage
private void loadNearbyPlaces(double lat, double lng) {
    List<Place.Field> fields = Arrays.asList(
            Place.Field.NAME,
            Place.Field.ADDRESS,
            Place.Field.PHOTO_METADATAS
    );
    FindCurrentPlaceRequest request = FindCurrentPlaceRequest.newInstance(fields);
    if (ActivityCompat.checkSelfPermission( context: this, Manifest.permission.ACCESS_FINE_LOCATION)
            != PackageManager.PERMISSION_GRANTED) {
        return;
    }
```

“loadNearbyPlaces” calls for the name, address and image of each place, that is close to the user.

Places API is called, using “SuccessListener” this will store the nerby places within “PlaceItem”.

```java
placesClient.findCurrentPlace(request).addOnSuccessListener( FindCurrentPlaceResponse response -> {
    List<PlaceItem> placeList = new ArrayList<>();
    //gets the info
    for (PlaceLikelihood placeLikelihood : response.getPlaceLikelihoods()) {
        Place place = placeLikelihood.getPlace();
        String name = place.getName();
        String address = place.getAddress();
        String photoUrl = null;
        PhotoMetadata metadata = null;
        if (place.getPhotoMetadatas() != null && !place.getPhotoMetadatas().isEmpty()) {
            metadata = place.getPhotoMetadatas().get(0);
        }
        Object type = null;
        //adds to place item
        placeList.add(new PlaceItem(name, address, photoUrl, metadata, type));
    }
    //shows it on recyclerveiw
    RecyclerView recyclerView = findViewById(R.id.recommendedRecycler);
    recyclerView.setLayoutManager(new LinearLayoutManager( context: this));
    recyclerView.setAdapter(new PlaceAdapter( context: this, placeList, placesClient));
})
    .addOnFailureListener( Exception e -> {
        Toast.makeText( context: this,  text: "Places API failed: " + e.getMessage(), Toast.LENGTH_SHORT).show();
    });
```

“Likehood” is used to get all places that are likely near the user. Then it gets the details, if there is not image it stays as “null”. If there is no image even though there is no “null” it makes sure to show an image “default_place_img”. “0” is used to get the first images of the place.

RecyclerVeiw is then used to show the places using “PlaceAdapter”.

If an error happens like not Wi-Fi or permissions for the API a message will be displaed.

Tabs Code

```java
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_tabs);

    tabType = getIntent().getStringExtra( name: "tab");
    if (tabType == null) {
        finish();
        return;
    }
```

The code for the TabsActivity is very similar, but the chages made where to filter what is shown to match the tab that has been clicked.

This makes sure that gets the tabs list to filter what it shows on the screen, this will be called “tabType”.

```java
private void loadPlaces(double lat, double lng, String tabType) {
    List<Place.Field> fields = Arrays.asList(
            Place.Field.NAME,
            Place.Field.ADDRESS,
            Place.Field.TYPES,
            Place.Field.PHOTO_METADATAS
    );
```

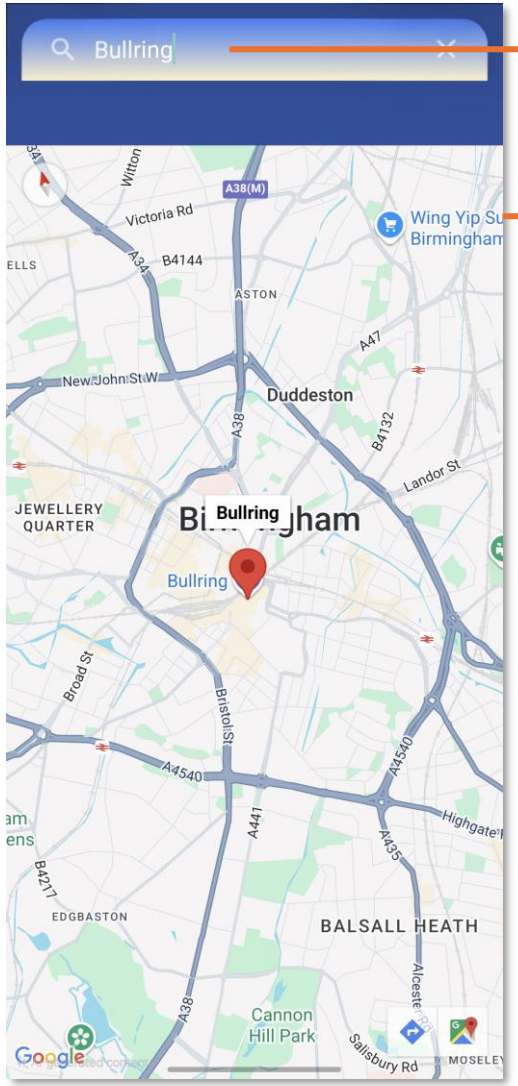
Same as home page, but “tabType” has been added so the information can be from the filtered list.

```java
placesClient.findCurrentPlace(request).addOnSuccessListener( FindCurrentPlaceResponse response -> {
    String[] allowedTypes = tabType.split( regex: "\\|");
    List<PlaceItem> placeList = new ArrayList<>();
    for (PlaceLikelihood likelihood : response.getPlaceLikelihoods()) {
        Place place = likelihood.getPlace();
        if (place.getTypes() != null) {
            for (Place.Type type : place.getTypes()) {
                String typeStr = type.toString().toLowerCase();
                for (String filter : allowedTypes) {
                    if (typeStr.equalsIgnoreCase(filter)) {
                        placeList.add(new PlaceItem(
                                place.getName(),
                                place.getAddress(),
                                photoUrl: null,
                                place.getPhotoMetadatas() != null && !place.getPhotoMetadatas().isEmpty()
                                        ? place.getPhotoMetadatas().get(0) : null,
                                filter
                        ));
                    }
                }
            }
        }
    }
```

As seen before each tabs button has a filtered list. Upon click the tabs, “.split” will turn the whole filter string into an array.

“Place.type” is used to see if the locations have types. To see if they match the types within the filters “String filter : allowed types” is used. Once its done just like the HomeActivity, it will add it to “PlaceItem” and shown through RecyclerVeiw.

Maps Page



The maps page can be accessed through pressing the “maps” icon, within the bottom navigation.
Here the user can see their current location, which will be updated live. So, in case of a tourist, they will know where they are at all times.

The user can also use the search bar to search up places such as “Bullring”, as shown in the image.

Maps Code - 1

```java
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_map);
    searchMap = findViewById(R.id.mapSearchBar);
    fusedLocationProviderClient = LocationServices.getFusedLocationProviderClient( activity: this);
    getLastLocation();
    searchMap.setOnQueryTextListener(new SearchView.OnQueryTextListener() {
        1 usage
        @Override
        public boolean onQueryTextSubmit(String query) {
            String location = searchMap.getQuery().toString();
            List<Address> addressList = null;

            if (location != null){
                Geocoder geocoder = new Geocoder( context: MapActivity.this);
                try {
                    addressList = geocoder.getFromLocationName(location,  maxResults: 1);
                } catch (IOException e) {
                    throw new RuntimeException(e);
                }

                Address address = addressList.get(0);
                LatLng latLng = new LatLng(address.getLatitude(), address.getLongitude());
                myMap.addMarker(new MarkerOptions().position(latLng).title(location));
                myMap.animateCamera(CameraUpdateFactory.newLatLngZoom(latLng,  v: 10));

            }
            return false;
        }
```

"FusedLocationProviderClient", is used like before to get the users last known location.

"searchMap" will allow the users to type in the locations. Then using "Geocoder" this will fine the coordinated of the searched location and limit it to only 1 result.

"LarLng" will get take the camera using "animateCamera" to where the marker was placed and zoom into the area at a level of 10.

Maps Code - 2

```java
private void getLastLocation() {
    if (ContextCompat.checkSelfPermission( context: this, Manifest.permission.ACCESS_FINE_LOCATION)
            != PackageManager.PERMISSION_GRANTED) {
        ActivityCompat.requestPermissions( activity: this,
                new String[]{Manifest.permission.ACCESS_FINE_LOCATION},
                FINE_PERMISSION_CODE);
    } else {
        Task<Location> task = fusedLocationProviderClient.getLastLocation();
        task.addOnSuccessListener( Location location -> {
            if (location != null) {
                currentLocation = location;

                SupportMapFragment mapFragment = (SupportMapFragment)
                        getSupportFragmentManager().findFragmentById(R.id.map);
                if (mapFragment != null) {
                    mapFragment.getMapAsync( onMapReadyCallback: MapActivity.this);
                }
            } else {
                Toast.makeText( context: this,  text: "Unable to get current location", Toast.LENGTH_SHORT).show();
            }
        });
    }
}
```

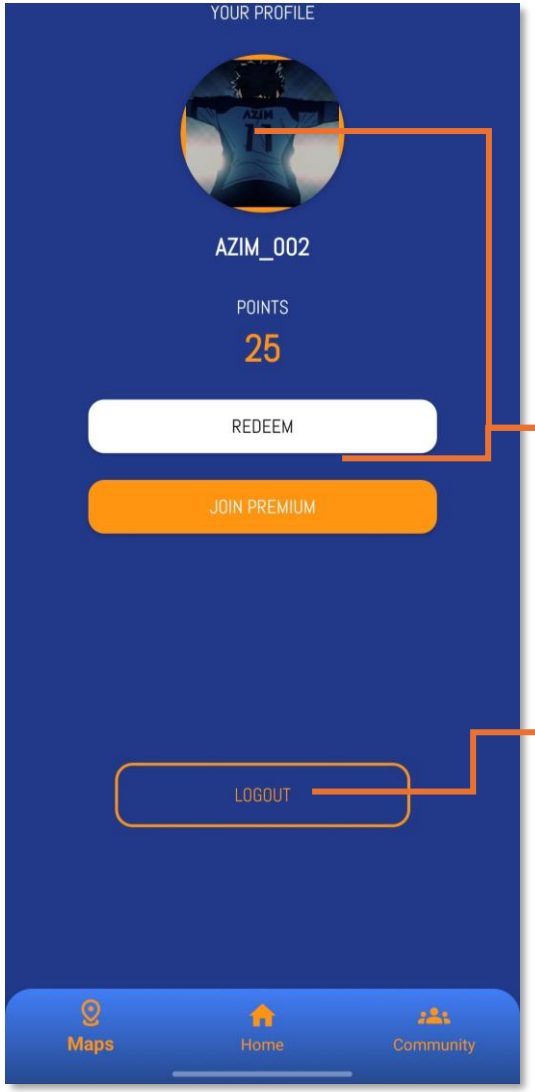
Here it gets the users permissions for the last known locations, once it does it loads the map “fragment”. This will then be ready for the “onMapReady” method.

```java
@Override
public void onMapReady(@NonNull GoogleMap googleMap) {
    if (currentLocation == null) {
        Toast.makeText( context: this,  text: "Location not ready", Toast.LENGTH_SHORT).show();
        return;
    }

    myMap = googleMap;
    LatLng userLocation = new LatLng(currentLocation.getLatitude(), currentLocation.getLongitude());
    myMap.addMarker(new MarkerOptions().position(userLocation).title( s: "My Location"));
    myMap.moveCamera(CameraUpdateFactory.newLatLngZoom(userLocation,  v: 15));
}
```

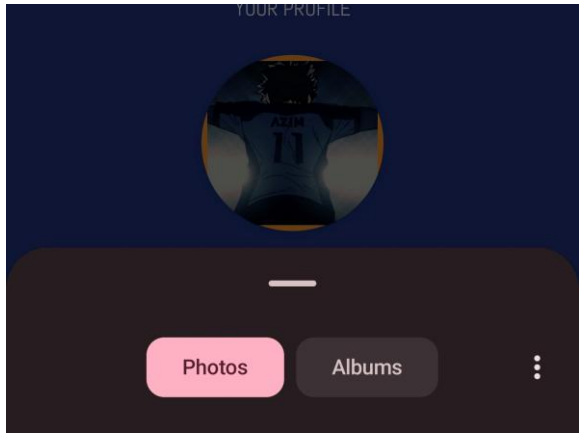
Once ready, “googleMap” is saved here. This will allow to use the marks and move the camera. “LatLng” will get the coordinates of the user and then a maker will be place, the camera will then zoom in at 15. this will show the users current location.

Profile Page

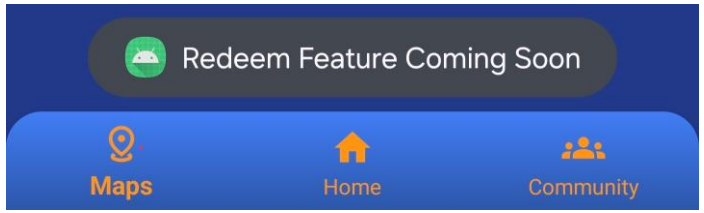


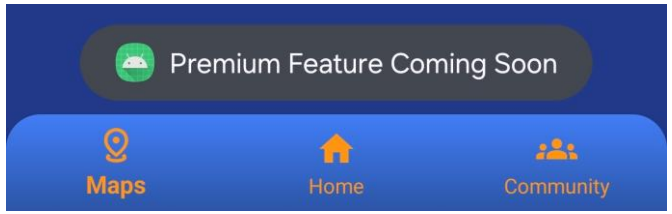
Upon the profile page, the user can click on their profile image and from there they will be able to change to whatever image they want.

The original plan was for the user to see their points gained from each location they visited and through that they can redeem gift cards. However, due to time constraints this was hardcoded in to show as an example. The “Redeem” and “Join Premium” buttons also do not work.



The “Logout” button will take the user back to the login screen. Also restarting the session, when the user closes the application.





Profile Code - 1

```java
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_profile);

    mAuth = FirebaseAuth.getInstance().getCurrentUser();
    storageRef = FirebaseStorage.getInstance().getReference( location: "profilePics");

    //variables xml
    usernameText = findViewById(R.id.usernameText);
    profileImage = findViewById(R.id.profileImage);
    logoutButton = findViewById(R.id.logoutButton);
    redeemButton = findViewById(R.id.redeemButton);
    premiumButton = findViewById(R.id.joinPremiumButton);

    //user info
    if (mAuth != null) {
        String name = mAuth.getDisplayName();
        Uri photoUri = mAuth.getPhotoUrl();
        if (name != null) {
            usernameText.setText(name.toUpperCase());
        }
        if (photoUri != null) {
            Glide.with( activity: this).load(photoUri).placeholder(R.drawable.defaultpfp)
                    .into(profileImage);
        }
    }
```

Using “mAuth” this will get the current user that is logged in, and creates them a directory to save images for their “profilepics”.

The user's information like their username will be displayed through “getDisplayName” from the firebase class, this will also display the users profile picture if they have set one.

Profile Code - 2

```java
//upload profile pic
profileImage.setOnClickListener( View v -> openImage());

//Logout
logoutButton.setOnClickListener( View v -> {
    FirebaseAuth.getInstance().signOut();
    startActivity(new Intent( packageContext: ProfileActivity.this, LoginActivity.class));
    finish();
});
```

"OnClickListenser" will allow the user to click on the profile picture and run the method "openImage"

Using firebase ".signOut", this will log the user out and redirect them to the login page. Once the lick on the logout button.

```java
private void openImage() {
    Intent intent = new Intent();
    intent.setType("image/*");
    intent.setAction(Intent.ACTION_GET_CONTENT);
    startActivityForResult(Intent.createChooser(intent, title: "Select profile picture"), PICK_IMAGE_REQUEST);
}
//image to firebase storage
@Override
protected void onActivityResult(int requestCode, int resultCode, @Nullable Intent data) {
    super.onActivityResult(requestCode, resultCode, data);

    if (requestCode == PICK_IMAGE_REQUEST && resultCode == RESULT_OK && data != null && data.getData() != null) {
        Uri img = data.getData();
        uploadProImage(img);
    }
}
```

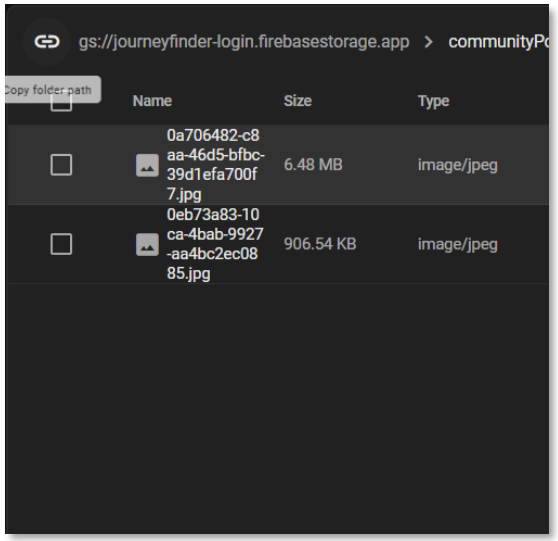
"openImage" will open the device photo files, this is where the user can pick an image for their profile. This is done through implicit intent, since it's asking the device for an image (data).
"onActivityResult" gets the request, if the user has picked an image and the images that was picked.

Profile Code - 3

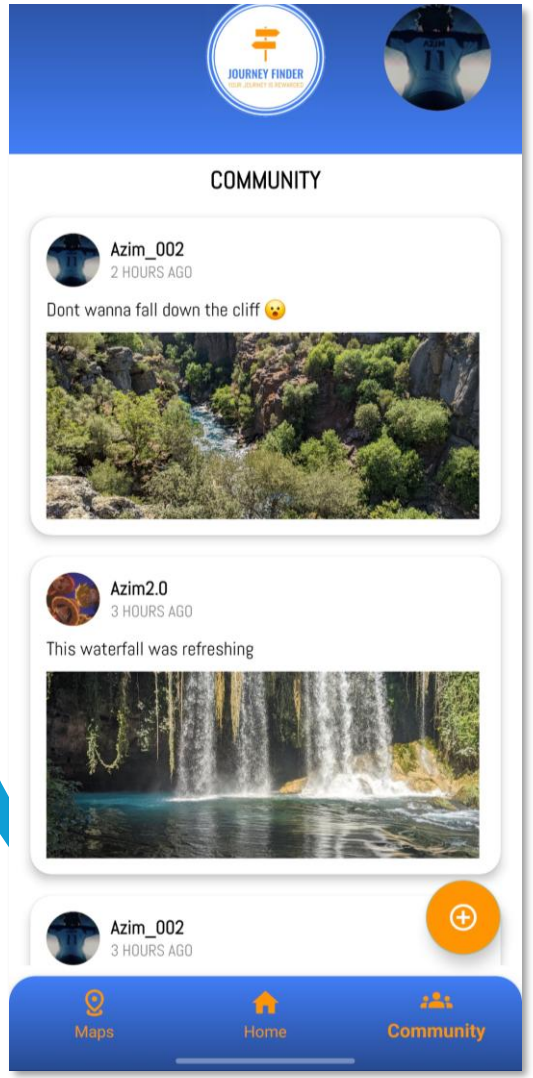
```java
private void uploadProImage(Uri imageUri) {
    if (mAuth == null) return;
    StorageReference userRef = storageRef.child( pathString: mAuth.getUid() + ".jpg");
    userRef.putFile(imageUri).addOnSuccessListener( TaskSnapshot taskSnapshot -> userRef.getDownloadUrl()
                    .addOnSuccessListener( Uri downloadUri -> {
                        //Save the download URL to the firebase profile
                        UserProfileChangeRequest profileUpdates = new UserProfileChangeRequest.Builder()
                                .setPhotoUri(downloadUri)
                                .build();
                        mAuth.updateProfile(profileUpdates).addOnCompleteListener( Task<Void> task -> {
                            if (task.isSuccessful()) {
                                Glide.with( activity: this).load(downloadUri).placeholder(R.drawable.defaultpfp)
                                        .into(profileImage);

                                Toast.makeText( context: this,  text: "Profile picture updated!", Toast.LENGTH_SHORT).show();
                            }
                        });
                    })).addOnFailureListener( Exception e ->
                    Toast.makeText( context: this,  text: "Upload failed: " + e.getMessage(), Toast.LENGTH_SHORT).show());
```

"uploadProfileImage" this method then checks if a user is signed in, if so, the image that they picked will be stored in the storage as a path linked to the users id. This then is updated in Firebase and can be called using "Glide" to display the new profile picture.

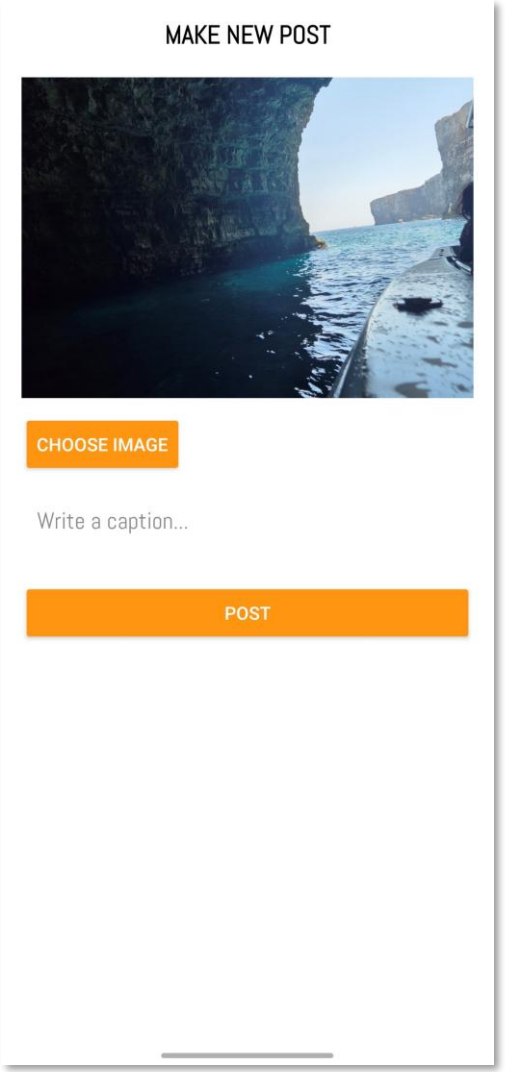


Community Page



The community page is where users can interact and share their experiences. This can be done by pressing the “plus” icon at the bottom, where they will be redirected to the “Make Post” page. After picking an image and writing a caption, the user will post and when going back to the community page, the post will be added through the RecyclerView. This will show who uploaded it and how long ago it was uploaded.

NOTE: for some reason after deleting the accounts through firebase and now making new ones, accounts cannot see others posts anymore. These images were taken during the first testing.



Community Code

```java
postRecycler = findViewById(R.id.postRecycler);
postRecycler.setLayoutManager(new LinearLayoutManager( context: this));
postList = new ArrayList<>();
postAdapter = new PostAdapter( context: this, postList);
postRecycler.setAdapter(postAdapter);

firestore = FirebaseFirestore.getInstance();

//to make new post button
FloatingActionButton NewPost = findViewById(R.id.NewPost);
NewPost.setOnClickListener( View v -> {
    startActivity(new Intent( packageContext: CommActivity.this, NewPostActivity.class));
});
```

Similar to the home page set up for RecylerView, here this is using “postItem” to list the items through “postAdapter”, which displays the viewing cards onto the page.

Due to wanting a button that doesn’t move while scrolling, “FloatingActionButton” was used to redrirect the user to the new post page.

```java
private void loadPosts() {
    firestore.collection( collectionPath: "community-posts").orderBy( field: "timePosted",
                com.google.firebase.firestore.Query.Direction.DESCENDING) Query
            .get() Task<QuerySnapshot>
            .addOnSuccessListener( QuerySnapshot querySnapshot -> {
                postList.clear();
                for (QueryDocumentSnapshot doc : querySnapshot) {
                    String username = doc.getString( field: "username");
                    String userProfileUrl = doc.getString( field: "userProfileUrl");
                    String caption = doc.getString( field: "caption");
                    String imageUrl = doc.getString( field: "imageUrl");
                    long time = doc.getLong( field: "timePosted") != null ? doc.getLong( field: "timePosted") : 0;

                    postList.add(new PostItem(username, userProfileUrl, formatTime(time), caption, imageUrl
                    ));
                }
                postAdapter.notifyDataSetChanged();
            });
}
1 usage
private String formatTime(long timestamp) {
    long now = System.currentTimeMillis();
    long diff = now - timestamp;
    long minutes = diff / (60 * 1000);
    long hours = minutes / 60;
    long days = hours / 24;

    if (days >= 1) return days + " DAY" + (days > 1 ? "S" : "") + " AGO";
    if (hours >= 1) return hours + " HOUR" + (hours > 1 ? "S" : "") + " AGO";
    if (minutes >= 1) return minutes + " MINUTE" + (minutes > 1 ? "S" : "") + " AGO";
    return "JUST NOW";
}
```

“loadPosts” will then grab the post that were stored within firestore and sort them from newest. Then using “postList” it is stored there.

“formatTime”, is so the time can be readable and displayed within the recyclerview cards.

New Post Code - 1

```java
currentUser = FirebaseAuth.getInstance().getCurrentUser();
storage = FirebaseStorage.getInstance().getReference( location: "communityPosts");
firestore = FirebaseFirestore.getInstance();

//Image
ActivityResultLauncher<Intent> imagePick = registerForActivityResult(
        new ActivityResultContracts.StartActivityForResult(),
        ActivityResult result -> {
            if (result.getResultCode() == RESULT_OK && result.getData() != null) {
                selectedImageUri = result.getData().getData();
                Glide.with( activity: this).load(selectedImageUri).into(postImagePreview);
            }
        });
```

“getReference” will direct to the Firebase storage which contains the communityposts. Then it will connect to the Firestore for the post contents.

Using “ActivityResultLauncher” this is an easy way to get the images the user want to post from their device.

```java
//Select Image
selectImageButton.setOnClickListener( View v -> {
    Intent intent = new Intent(Intent.ACTION_PICK, MediaStore.Images.Media.EXTERNAL_CONTENT_URI);
    imagePick.launch(intent);
});

//Upload Post
uploadPostButton.setOnClickListener( View v -> {
    String caption = captionInput.getText().toString();

    if (selectedImageUri == null) {
        Toast.makeText( context: this,  text: "Please select an image", Toast.LENGTH_SHORT).show();
        return;
    }
    if (caption.isEmpty()) {
        Toast.makeText( context: this,  text: "Caption cannot be empty", Toast.LENGTH_SHORT).show();
        return;
    }
    startActivity(new Intent( packageContext: NewPostActivity.this, CommActivity.class));
    finish();
    uploadImage(caption);
});
```

Just like the profile page, this opens the image picker so the user can choose an image. This wis done using glide, to show the image.
As for the “uploadButton” this again uses “OnClickListener” and checks if both the captions and image have been completed. The it runs “uploadImage” method.

New Post Code - 2

```java
private void uploadImage(String caption) {
    String fileName = UUID.randomUUID().toString() + ".jpg";
    StorageReference imageRef = storage.child(fileName);

    imageRef.putFile(selectedImageUri) UploadTask
            .addOnSuccessListener( TaskSnapshot taskSnapshot ->
                    imageRef.getDownloadUrl().addOnSuccessListener( Uri downloadUri -> {
                        savePost(downloadUri.toString(), caption);
                    })) StorageTask<TaskSnapshot>
            .addOnFailureListener( Exception e ->
                    Toast.makeText( context: this,  text: "Upload failed: " + e.getMessage(), Toast.LENGTH_SHORT).show());
}
```

Using “UUID” will randomly id the image file, so images cannot be overwritten in case they have the same name.
This is then uploaded in the storage using “putFile”, this will then be saved to the firestore, within the “community-posts” database.

```java
private void savePost(String imageUrl, String caption) {
    String userName = currentUser.getDisplayName() != null ? currentUser.getDisplayName() : "no name";
    String profileUrl = currentUser.getPhotoUrl() != null ? currentUser.getPhotoUrl().toString() : "";

    Map<String, Object> post = new HashMap<>();
    post.put("username", userName);
    post.put("userProfileUrl", profileUrl);
    post.put("caption", caption);
    post.put("imageUrl", imageUrl);
    post.put("timePosted", System.currentTimeMillis());

    firestore.collection( collectionPath: "community-posts").add(post).addOnSuccessListener(
            DocumentReference documentReference -> {
        Toast.makeText( context: this,  text: "Post uploaded", Toast.LENGTH_SHORT).show();
        finish();
    }).addOnFailureListener( Exception e -> Toast.makeText( context: this,  text: "Failed to save post: " + e.getMessage(),
            Toast.LENGTH_SHORT).show());
}
```

The “savePost” methos will make a “HashMap”, which can store database keys and values as pairs. This will be able to display the username, profile picture, time of post and the image and caption.
Due, to the all posts being stored publicly within the firestore database, posed from other account should be seen.

Challenges

- During the development of this application, there were a couple challenges like figuring out how to get the search bar working within the home page, so I can filter through the nearby areas. Due to time constrained this wasn't implemented, and when tired the app would crash upon making a spelling error.
- Another major challenge was using Firebase Firestore for the live database, when making a post. When this was finally set in to work, login in as other users would show posts from other accounts, as shown within the community page. However, upon deleting the test accounts and adding in new ones this feature does not work. This could be due to the rules set for the Firestore.

Future Improvements

- For improvements in the future could be, adding a working reward system.
- Implementing a feature where once the user click on the recommended card for a location, it will redirect them to the maps page, with a route showing how to get the. This was initially though of but again due to time constraints, this couldn't be done.
- Implement the initial plan for a monthly subscription for business owners, which will boost there shop up the recommendations.
- Since this application is for traveling, having a wearable version of it on a watch could be considered.
- Also, many of these features were implemented following Google documentations, android developers and many YouTube videos. So, for the future maybe not follow YouTube videos, but find things out myself.

References

- Firebase. (n.d.). *Authenticate with Firebase using Password-Based Accounts on Android*. [online] Available at: https://firebase.google.com/docs/auth/android/password-auth.
- Firebase. (2025). *Get Started with Firebase Authentication on Android*. [online] Available at: https://firebase.google.com/docs/auth/android/start#java_4
- Firebase. (2025). *Manage Users in Firebase*. [online] Available at: https://firebase.google.com/docs/auth/android/manage-users#update_a_users_profile
- Google for Developers. (2025). *Place Types*. [online] Available at: https://developers.google.com/maps/documentation/places/web-service/legacy/supported_types
- Android Developers. (2019). *Intents and Intent Filters | Android Developers*. [online] Available at: https://developer.android.com/guide/components/intents-filters.
- Firebase. (2025). *Order and limit data with Cloud Firestore | Firebase*. [online] Available at: https://firebase.google.com/docs/firestore/query-data/order-limit-data#java
- Android Developers. (n.d.). *Getting a result from an activity*. [online] Available at: https://developer.android.com/training/basics/intents/result.
- Firebase. (n.d.). *Upload files with Cloud Storage on Android*. [online] Available at: https://firebase.google.com/docs/storage/android/upload-files.